

GITHUB 3 - Branch

Auteur

Nom : [DEMAISON Camille]

Voici une documentation nous permettant d'utiliser GitHub.

GitHub est un outils de versioning qui nous permets de centraliser nos ressources et de créer différentes versions tout au long du projet.

Branches et travail en parallèle

Pour travailler sur des parties critiques du projet, qui pourraient casser le fonctionnement de systèmes on peut créer des branches. Cela permet de créer une version parallèle et donc tout push sur cette branch n'affectera pas les autres branches.

Pour créer une branche :

```
git branch name_of_the_branch
```

Pour changer de branche : (modifie directement les dossiers et fichiers dans le dossier Github)

```
git checkout name_of_the_branch
```

Ensuite on peut commit comme avant sur notre branche comme ceci :

```
git push -u origin new-branch
```

Pour supprimer la branche locale : (ATTENTION, qu'en local tant qu'on n'a pas push)

```
git branch -d name_of_the_branch
```

Pour supprimer directement la branche distante :

```
git push REMOTE-NAME :BRANCH-NAME
```

```
# Exemple :
```

```
git push https://github.com/Bioubiou666/projet_L3.git :test
```

Et pour fusionner deux branches on utilise `merge` . Il faut d'abord se placer dans la branch qui prend la fusion, puis fusionner la branche voulu.

```
git checkout stay_branch  
git pull origin stay_branch
```

```
# Fusionne en local
```

```
git merge feature_branch
```

```
#
```

```
git push origin stay_branch
```

Warning

Il est possible qu'il y ai des conflits de merge.

Par exemple si la même ligne de code a été modifiée dans `feature_branch` et `main` , Git ne sait pas quelle version garder.

Git demandera de **résoudre manuellement les conflits**, en éditant les fichiers touchés.

Exemple de conflit dans un fichier (`index.js`) :

```
<<<<<< stay_branch
console.log("Hello from main");
=====
console.log("Hello from feature-branch");
>>>>>> feature-branch
```

Il faudra conserver la partie voulue dans le fichier puis sauvegarder. Une fois les conflits gérés il faudra push car il n'avait pas pu aboutir :

```
# Il est possible de ne renseigner qu'un fichier au lieu de tous lors du
add
git add .
git commit -m "Merge - Résolution du conflit"
git push origin stay_branch
```

Attention avant de push vers le serveur, il faut toujours vérifier que le merge n'a pas tout cassé si vous avez du résoudre des conflits lors du merge.

Petit récapitulatif et commandes optionnelles



GitHub Cheat Sheet

Versionner son travail

Versionner en local

git init	initialise le dépôt (se mettre sur le bon dossier), mieux à faire depuis Github.com
git add .	ajoute toutes les modifications (le . symbolise tout)
git commit -m "explication"	créer un nouveau commit, git add pousse les fichiers en zone d'index, git commit les sauvegarde réellement dans un nouveau commit

Gérer les commits

git log	liste des commits
git log -n2	affiche les 2 derniers commits
git show sha-1	voir commit spécifique (cliquer molette souris pour coller)
git checkout sha-1	remettre la version du sha-1
git checkout main	remettre version la plus récente

Versionner sur un dépôt distant

git clone lien-github.com	récupérer travail depuis dépôt distant
git push -u origin main	pousse les modifications vers serveur
git push -f origin main	pousse de force des modifications (à manipuler avec précaution)

Naviguer dans Git Bash

pwd	savoir dans quel dossier je suis
mkdir "dossier"	créer un dossier (Make Directory)
touch fichier.txt	créer fichier
ls	liste le dossier courant
ls -la	liste tout plus précisément que ls
cd dossier	aller dans le dossier (Change Directory)
cd ..	Remonter d'un dossier

Initialisation de Git

git config --global user.name	"Mon Nom"
git config --global user.email	mon@mail.com
git config --global --list	Affiche nom et mail

Autres commandes

git status	état du fichier
git diff	affiche les modifs avant commit

Commit son projet sur Github

git add .
git commit -m "message"
git push -u origin main

